

MercalIntelligence



Plataforma de inteligencia de precios sobre el catálogo de Mercadona con scraping diario, ML/DL, NLP y visualización en tiempo real.

Descripción

MercalIntelligence es un sistema integral de monitorización, análisis y predicción de precios del catálogo online de Mercadona. El proyecto aborda un problema real del consumidor y del analista de retail: la falta de herramientas automatizadas para rastrear la evolución de más de 5.000 productos a lo largo del tiempo, detectar anomalías en precios, identificar equivalencias entre marcas propias y comerciales, y anticipar cambios futuros.

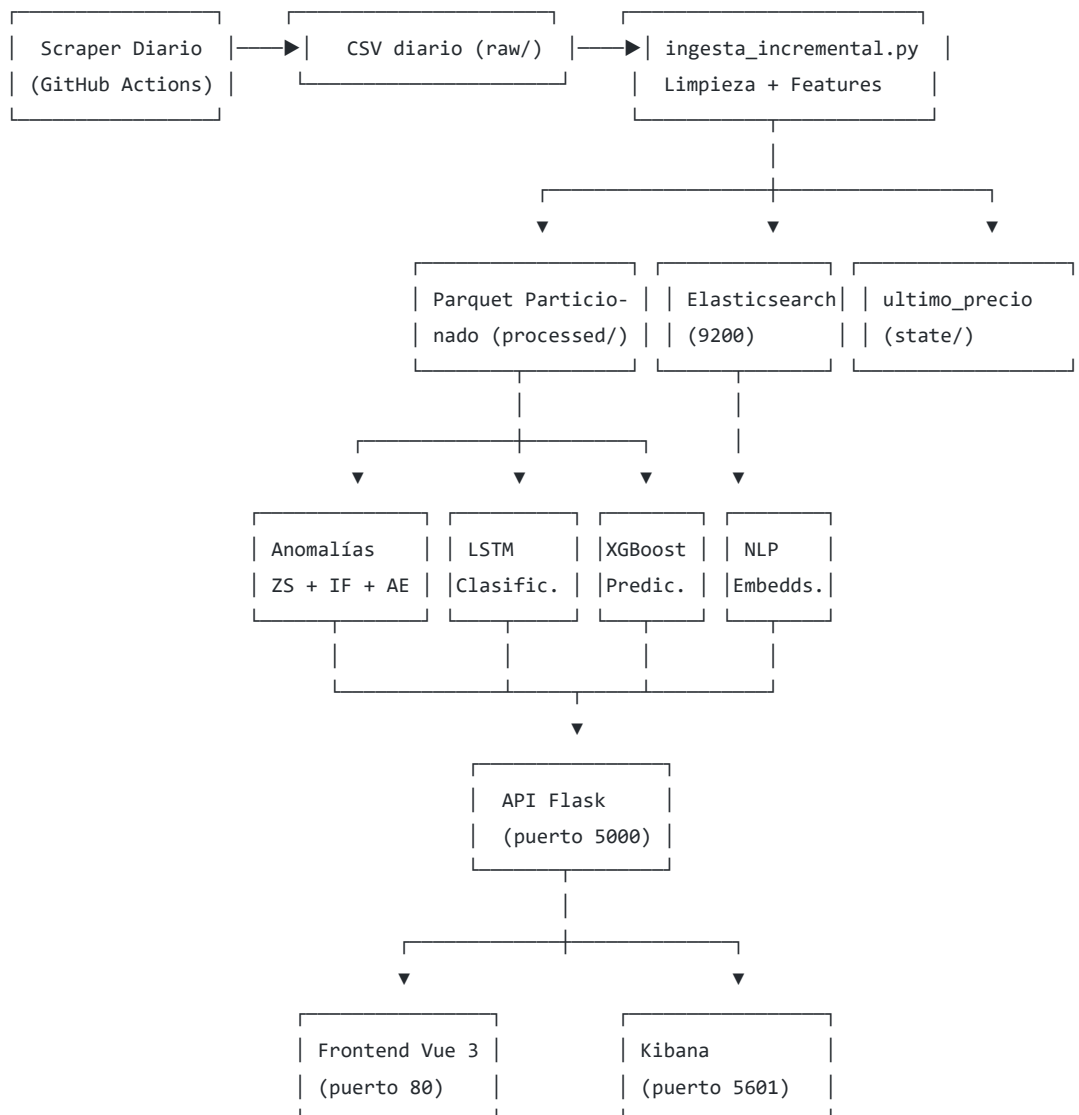
El sistema ingiere diariamente los datos de un scraper externo ejecutado mediante GitHub Actions, los procesa a través de un pipeline ETL incremental que genera un Data Lake en formato Parquet particionado por fecha, y los indexa en Elasticsearch para su exploración visual en Kibana. Sobre esta base de datos se ejecutan módulos de Machine Learning y Deep Learning que incluyen detección de anomalías con tres enfoques complementarios (Z-Score, Isolation Forest y Autoencoder LSTM), un clasificador LSTM para predecir cambios de precio a 7 días, un regresor XGBoost que estima el precio futuro, y un pipeline NLP basado en sentence-transformers que empareja automáticamente productos de marca propia con sus equivalentes comerciales.

El frontend en Vue 3 permite al usuario construir cestas de la compra personalizadas, calcular un IPC personalizado ponderado por consumo real, visualizar predicciones de coste futuro, consultar alertas de anomalías y shrinkflation, y explorar las equivalencias de marca con su brecha de precio. Todo el stack se despliega con Docker Compose en cuatro servicios independientes (Elasticsearch, Kibana, API Flask y Frontend Nginx).

El valor diferencial del proyecto reside en la convergencia inter-modular: tres modelos independientes (LSTM, XGBoost y Autoencoder) validan mutuamente sus hallazgos, el LSTM alimenta al XGBoost como feature, y la degradación temporal del XGBoost coincide exactamente con los picos de anomalías del Autoencoder, confirmando que los cambios detectados son genuinos.



Arquitectura



El principal problema con las tablas horizontales en Markdown cuando manejan rutas de archivos extensas y múltiples *outputs* es que saturan la pantalla, provocan un scroll horizontal incómodo o rompen el diseño con saltos de línea caóticos.

Aquí tienes **dos alternativas** optimizadas para que tu documentación se vea limpia, legible y profesional.

Módulos de Machine Learning

Detección de Anomalías

- **Baseline (Z-Score Rolling)** 14d, 2.5σ
- **Código:** `src/ml/anomalias_zscore.py`
- **Output:** `data/anomalias/zscore_resultados.parquet`
- **Machine Learning (Isolation Forest)** 200 árboles, contam=0.5%
- **Código:** `src/ml/anomalias_isolation_forest.py`
- **Outputs:** *  `data/anomalias/if_resultados.parquet`
-  `models/isolation_forest.pkl`
- **Deep Learning (Autoencoder LSTM)** ventana=14d, P99
- **Código:** `src/ml/anomalias_autoencoder.py`
- **Outputs:** *  `data/anomalias/ae_resultados.parquet`
-  `models/autoencoder_lstm.keras`

Modelos de Predicción

- **Predicción de Cambio (LSTM Clasificador Binario)** 14d → 7d
- **Notebook:** `notebooks/lstm_clasificador_colab.ipynb`
- **Outputs:** *  `data/predicciones/lstm/lstm_resultados.parquet`
-  `models/lstm_clasificador.keras`
- **Predicción de Precio (XGBoost Regresor)** 21 features, horizonte 7d
- **Código:** `src/ml/xgboost_prediccion.py`
- **Outputs:** *  `models/xgboost_precio.pkl`
-  `models/xgboost_encoders.pkl`

Procesamiento, NLP y Métricas de Negocio

- **Equivalencias NLP:** `sentence-transformers` + similitud coseno
- **Código:** `src/ml/nlp_embeddings.py`
- **Output:** `data/nlp/equivalencias.parquet`
- **Segmentación:** K-Means (K=5, 7 features, PCA)
- **Notebook:** `notebooks/clustering_kmeans.ipynb`
- **Output:** `data/clustering/`
- **Shrinkflation:** Detección por correlación precio/medida
- **Código:** `src/etl/detector_shrinkflation.py`
- **Output:** `data/shrinkflation/alertas.parquet`
- **Rotación de Catálogo:** Altas/bajas con *burn-in* y confirmación
- **Código:** `src/etl/detector_catalogo.py`
- **Outputs:** *  `data/catalogo/nuevos.parquet`
-  `descatalogados.parquet`

Stack tecnológico

Backend	Frontend / Infraestructura
Python 3.11	Vue 3 + Composition API
pandas + PyArrow	Vite
Elasticsearch 7.17	ECharts (visualizaciones)
Flask + Flask-CORS	Axios (HTTP client)
TensorFlow / Keras	Nginx (proxy inverso)
XGBoost	Docker + Docker Compose
scikit-learn	Kibana 7.17
sentence-transformers (paraphrase-multilingual-MiniLM-L12-v2)	pnpm (gestor de paquetes)
NumPy	GitHub Actions (scraper)
watchdog (vigilancia de ficheros)	Google Colab (entrenamiento GPU)

Estructura del proyecto

```

mercaintelligence/
├── data/                                # Data Lake del proyecto
│   ├── raw/                            # CSVs diarios originales del scraper
│   ├── processed/                      # Parquet particionado por fecha (fecha=YYYY-MM-DD/)
│   ├── state/                          # Estado transaccional (ultimo_precio.parquet)
│   ├── anomalias/                      # Resultados de los 3 detectores de anomalías
│   ├── predicciones/                  # Predicciones LSTM y XGBoost
│   ├── nlp/                           # Equivalencias semánticas marca propia ↔ comercial
│   ├── clustering/                    # Resultados de segmentación K-Means
│   ├── catalogo/                      # Productos nuevos y descatalogados
│   └── shrinkflation/                 # Alertas de reduflación
├── models/                             # Modelos entrenados (.pkl y .keras)
├── src/
│   ├── etl/                            # Pipeline ETL: ingesta, consolidación, indexación
│   │   ├── consolidar_historico.py      # Construcción inicial del parquet maestro
│   │   ├── ingesta_incremental.py       # Ingesta diaria con watchdog
│   │   ├── master_ingest.py             # Orquestador maestro de indexación en ES
│   │   ├── detector_catalogo.py         # Detección de altas y bajas del surtido
│   │   ├── detector_shrinkflation.py     # Detección de reduflación
│   │   ├── es_utils.py                  # Utilidades de conexión a Elasticsearch
│   │   ├── indexar_historico_es.py       # Indexación del histórico en ES
│   │   ├── indexar_anomalias_es.py      # Enriquecimiento de ES con scores de anomalía
│   │   ├── indexar_equivalencias_es.py   # Indexación de equivalencias NLP en ES
│   │   ├── indexar_ipc_es.py            # Indexación de series IPC en ES
│   │   └── migrar_a_particionado.py      # Migración de monolítico a particionado
│   ├── ml/                             # Modelos de Machine Learning y Deep Learning
│   │   ├── anomalias_zscore.py          # Detector Z-Score Rolling
│   │   ├── anomalias_isolation_forest.py # Detector Isolation Forest
│   │   ├── anomalias_autoencoder.py     # Autoencoder LSTM (inferencia local)
│   │   ├── nlp_embeddings.py            # Pipeline NLP de equivalencias semánticas
│   │   └── xgboost_prediccion.py        # Regresor XGBoost de precio futuro
│   └── api/
│       └── app.py                       # API Flask con todos los endpoints
├── frontend/                           # Aplicación Vue 3 + Vite
│   └── src/
│       ├── views/                      # Vistas: Home, IPC, Anomalías, Buscador, Marcas, etc.
│       ├── services/api.js             # Capa de abstracción para llamadas a la API
│       ├── components/                 # Componentes Vue reutilizables
│       └── router/                     # Enrutamiento SPA
├── notebooks/                          # Notebooks de entrenamiento y análisis
├── docs/                               # Documentación y análisis detallados
│   └── analisis/                       # Documentos de análisis por módulo
├── dashboards/                         # Exportación de dashboards Kibana (NDJSON)
├── docker-compose.yml                  # Orquestación de 4 servicios
├── Dockerfile.api                      # Imagen de la API Flask
├── Dockerfile.frontend                 # Multi-stage build: Node → Nginx
├── nginx.conf                          # Configuración de proxy inverso
└── requirements.txt                    # Dependencias Python

```

IPC personalizado — Metodología

El IPC personalizado es una de las funcionalidades centrales del sistema. A diferencia del IPC oficial (que pondera por consumo agregado de la población), MercaIntelligence permite al usuario construir una cesta con productos específicos y cantidades mensuales reales, calculando un índice ponderado por su consumo personal:

1. **Fecha base:** primera fecha disponible del dataset (2025-11-03)
2. **Pesos automáticos:** $\text{gasto}_i = \text{precio_base}_i \times \text{cantidad_mensual}_i \rightarrow \text{peso}_i = \text{gasto}_i / \text{gasto_total}$
3. **Índice por producto:** $\text{índice}_t = \text{precio}_t / \text{precio_base} \times 100$
4. **IPC de la cesta:** $\text{IPC}(t) = \sum [\text{peso}_i \times \text{índice}_i(t)]$

El sistema incluye 6 perfiles predefinidos (familiar, estudiante, vegano, deportista, pareja y una cesta personal del autor) con productos y cantidades reales. La predicción de coste futuro combina la probabilidad de cambio del LSTM con la tendencia histórica de cada producto.

Instalación y uso

Opción A — Docker (recomendado)

```
git clone https://github.com/DanielPMdev/mercaintelligence.git
cd mercaintelligence
docker-compose up -d
```

Servicio	URL	Descripción
Frontend	http://localhost	Interfaz de usuario Vue 3
API Flask	http://localhost:5000	Endpoints REST
Kibana	http://localhost:5601	Dashboards de exploración
Elasticsearch	http://localhost:9200	Motor de búsqueda e indexación

Opción B — Desarrollo local

```
# 1. Clonar y crear entorno
git clone https://github.com/DanielPMdev/mercaintelligence.git
cd mercaintelligence
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate # Linux/macOS

# 2. Instalar dependencias Python
pip install -r requirements.txt

# 3. Consolidar histórico (solo la primera vez)
python src/etl/consolidar_historico.py

# 4. Levantar Elasticsearch y Kibana
docker-compose up -d es01 kibana

# 5. Indexar histórico y anomalías en Elasticsearch
python src/etl/master_ingest.py

# 6. Lanzar la API Flask
python src/api/app.py

# 7. Lanzar el frontend (en otra terminal)
cd frontend
pnpm install
pnpm dev
```




Resultados obtenidos

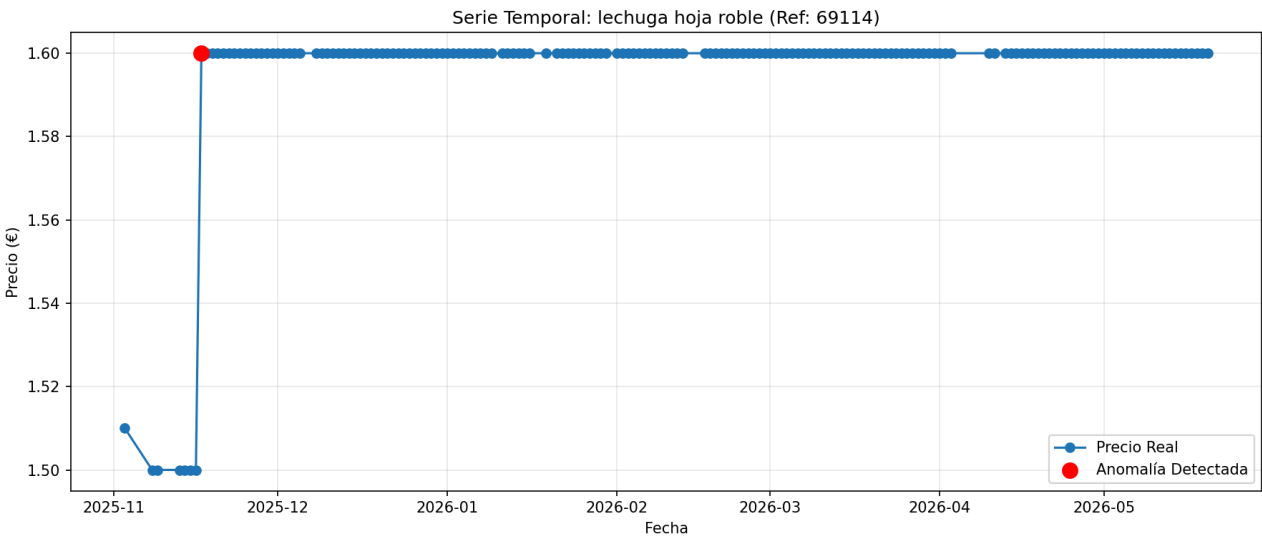
Módulo	Métrica	Resultado
Z-Score (14d, 2.5σ)	Tasa de anomalía	0.50% · 3,813 alertas · 1,683 productos
Isolation Forest	Tasa de anomalía	0.50% · 3,818 alertas · 59 productos
Autoencoder LSTM	Tasa de anomalía	1.00% · 6,929 alertas · 1,300 productos
Jaccard ZS ↔ IF ↔ AE	Solapamiento entre métodos	0.006 / 0.000 / 0.005 — métodos complementarios
LSTM clasificador	AUC-ROC en test	0.893 vs 0.771 baseline LR
XGBoost regresor	MAPE en cambios reales	10.75% vs 11.22% baseline naive (+4.2% mejora)
NLP sentence-transformers	Similitud media	0.844 · 924 equivalencias · brecha mediana +49.0%
K-Means	Silhouette Score (K=5)	0.441 · 76.0% varianza PCA 2D
Shrinkflation	Casos detectados	20 alertas · 100% producto fresco · severidad media 12.49
Catálogo	Rotación detectada	615 nuevos · 717 descatalogados · pico marzo (+157)

Ver análisis detallados por módulo en [docs/analisis/](#) .

Detección de anomalías

Método	Evaluadas	Anomalías	Tasa	Productos afectados	Jaccard vs ZS	Jaccard vs IF
Z-Score Rolling (14d, 2.5σ)	763,435	3,813	0.50%	1,683	—	—
Isolation Forest (200 árboles, contam=0.5%)	763,435	3,818	0.50%	59	0.006	—
Autoencoder LSTM (ventana=14d, P99)	692,816	6,929	1.00%	1,300	0.000	0.005

La complementariedad extrema (Jaccard ≈ 0) confirma que cada método captura una dimensión de anomalía completamente distinta.



Explicación: Ejemplo visual del rolling Z-Score sobre una serie temporal real de precios. El gráfico muestra cómo el score dinámico detecta de forma efectiva el cambio repentino en la serie temporal y resalta las anomalías con círculos rojos en los puntos de precio real.

Ejemplo de detección Z-Score: subida brusca de precio identificada como anomalía (punto rojo) sobre la serie temporal real de un producto.

Predicción de precios

Métrica	XGBoost	Baseline Naive
MAE (€)	0.1304	0.0230
MAPE (%)	1.29%	0.46%
R ²	0.7993	0.9984

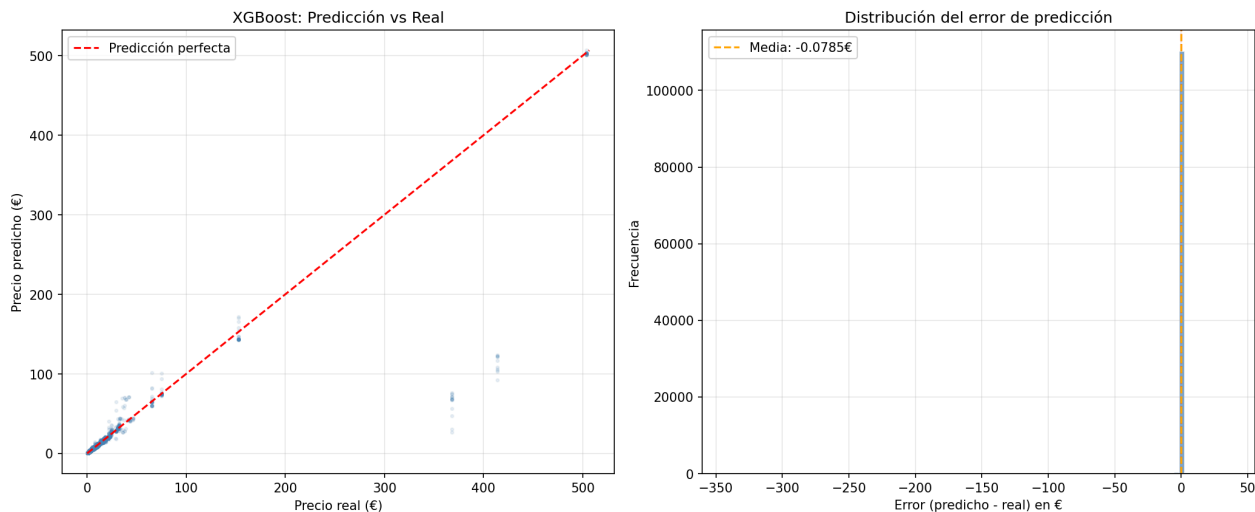
El 95.9% de los precios no cambian en 7 días, haciendo que la persistencia sea casi óptima. En el 4.1% con cambio real, XGBoost mejora el MAPE en un +4.2%.

Walk-forward validation (3 folds expansivos):

Fold	Train	Test	MAE (€)	R ²	MAPE (%)
1	275,940	91,086	0.0448	0.9984	0.86%
2	367,026	90,568	0.1110	0.8496	1.08%
3	457,594	90,126	0.1360	0.8042	1.36%
Media	—	—	0.0973	0.8841	1.10%

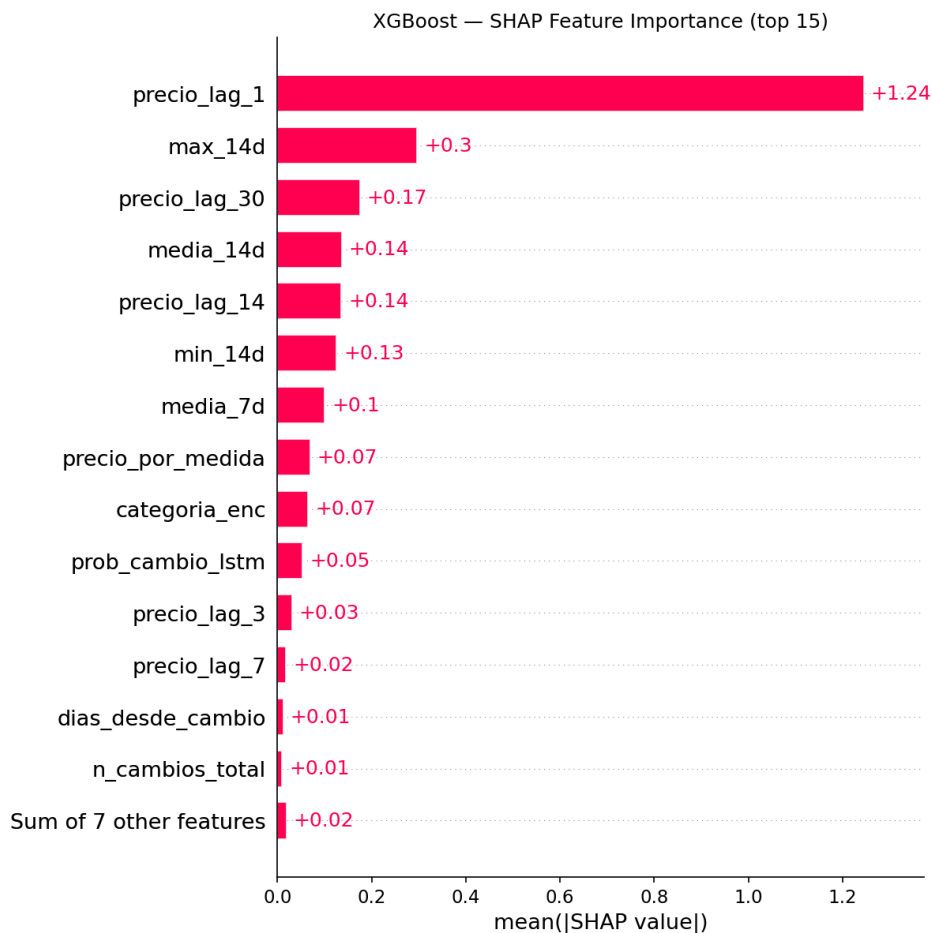
La degradación temporal (R^2 0.99 $\rightarrow$ 0.80) coincide con los picos de anomalías detectados por el Autoencoder en abril-mayo 2026, validando de forma cruzada ambos módulos.

XGBoost Regressor — Horizonte 7 días



Explicación: Gráfica de dispersión comparando el precio real vs la predicción del modelo XGBoost (t+7) en el gráfico izquierdo. La proximidad de los puntos a la diagonal representa alta precisión. El histograma del error en gráfico derecho muestra la media y dispersión del error de predicción en euros.

Dispersión predicción vs precio real (izq.) y distribución del error de predicción en € (dcha.). La mayoría de predicciones se ajustan a la diagonal, con errores notables solo en productos de gama alta (>300€).



Explicación: Importancia de las variables basada en valores SHAP. Muestra el impacto medio absoluto sobre las predicciones de precios a 7 días. Las variables históricas de precio y las agregaciones temporales dominan la toma de decisiones.

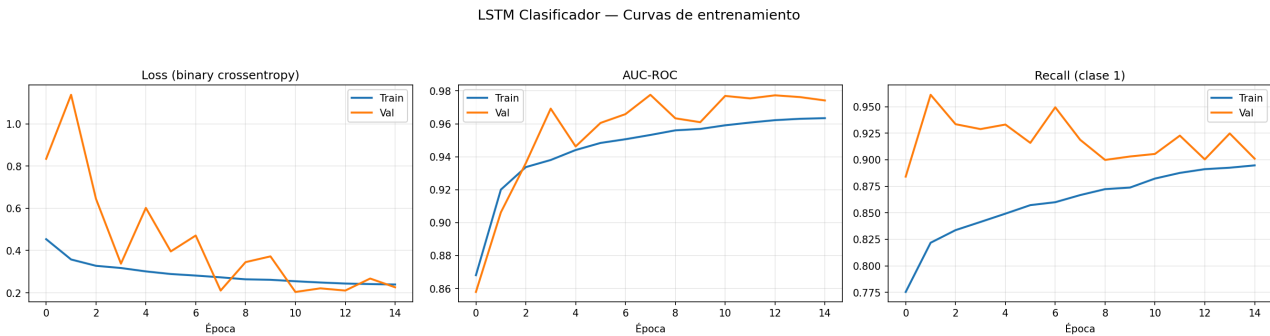
Importancia SHAP de las 15 features principales. `precio_lag_1` domina con un impacto medio de +1.24, seguido del máximo a 14 días y lags temporales. Nótese `prob_cambio_lstm` en el top 11, confirmando la

integración inter-modular.

LSTM Clasificador

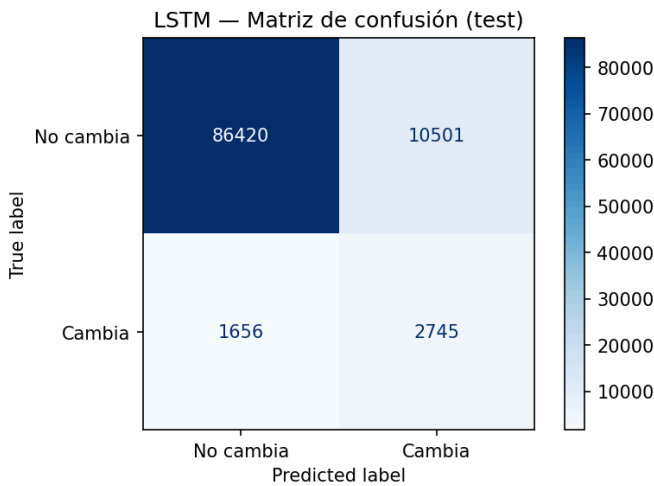
Modelo	Precision	Recall	F1-score	AUC-ROC
LSTM	0.207	0.624	0.311	0.893
Regresión Logística (baseline)	0.156	0.279	0.201	0.771

El LSTM genera 658,373 predicciones históricas cubriendo 4,858 productos, con 31,054 alertas de cambio de precio.



Explicación: Evolución de la pérdida y métricas clave (AUC, Recall) durante el entrenamiento del clasificador LSTM. La convergencia de las curvas de entrenamiento y validación indica un buen aprendizaje sin sobreajuste aparente, logrando un recall aceptable en la clase minoritaria

Curvas de entrenamiento: convergencia de Loss, AUC-ROC (0.98 en validación) y Recall (clase 1) a lo largo de 15 épocas. Early Stopping restauró los pesos de la época 8.

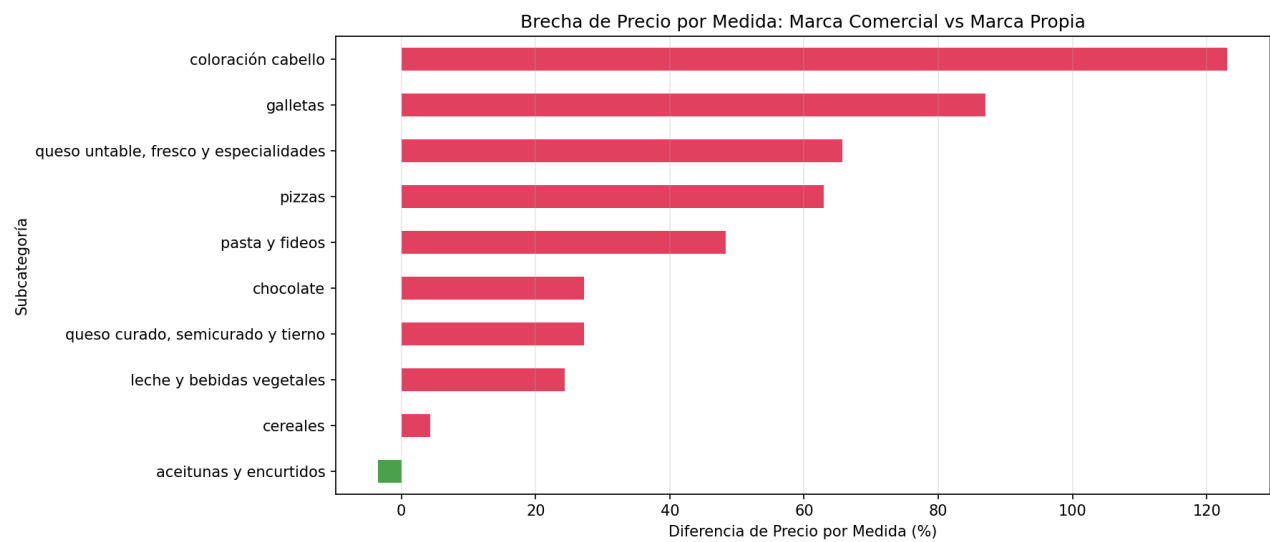


Explicación: Matriz de confusión del modelo LSTM en el conjunto de prueba utilizando el umbral optimizado. Muestra la capacidad del modelo para clasificar correctamente ambas clases, evaluando la tasa de verdaderos positivos (cambios detectados) frente a los falsos positivos (alarmas falsas).

Matriz de confusión en test: el LSTM detecta 2,745 de los 4,401 cambios reales (Recall 62.4%), generando 10,501 falsos positivos — un trade-off controlado por la optimización del umbral.

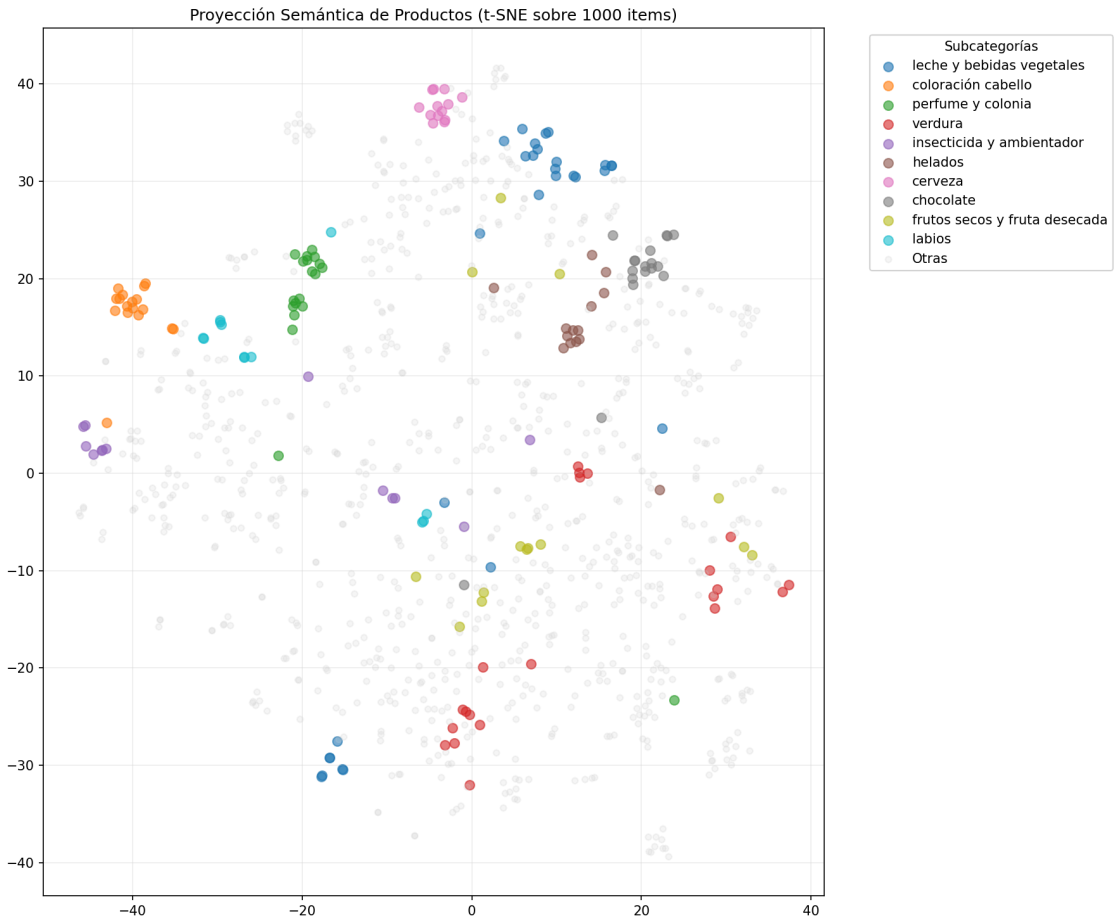
Equivalencias NLP

Métrica	Valor
Equivalencias top-1 generadas	924
Similitud media coseno	0.844
Pares comparables (misma unidad)	892
Brecha mediana marca propia vs comercial	+49.0%



Explicación: Distribución de la brecha de precios porcentual entre productos de marca comercial de fabricante frente a sus alternativas de marca blanca de Mercadona. Valores superiores a 0 indican que la marca comercial es proporcionalmente más cara.

Brecha de precio por medida (%) entre marca comercial y marca propia en las 10 subcategorías con más pares. Coloración de cabello lidera con +120%, mientras que aceitunas y encurtidos es la única categoría donde la marca propia es más cara.



Explicación: Proyección bidimensional t-SNE de los embeddings de alta dimensión del catálogo de productos. La agrupación visual por colores demuestra la capacidad del modelo para estructurar categorías semánticas coherentes en el espacio latente.

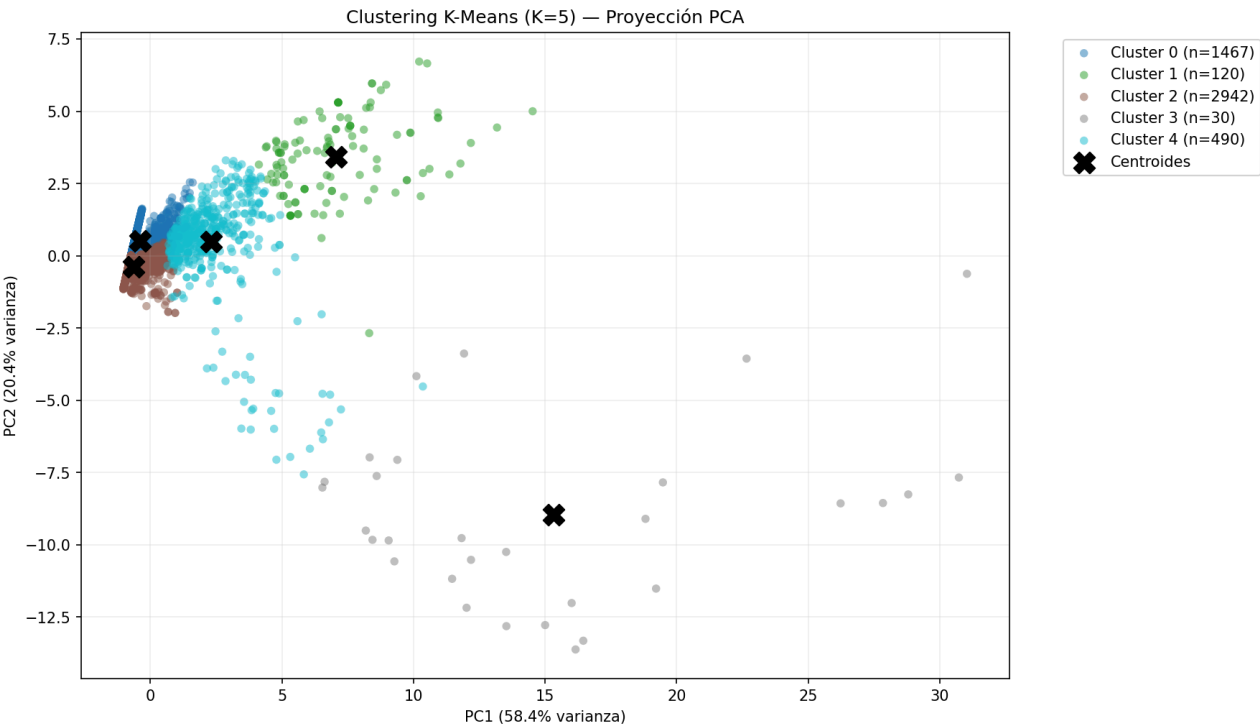
Proyección t-SNE de los embeddings del catálogo. La agrupación visual por subcategorías (coloración cabello, cerveza, chocolate, perfumería) valida la capacidad del modelo para estructurar categorías semánticas coherentes.

Segmentación K-Means

Métrica	Valor
Productos segmentados	5,049
Clusters (K)	5
Silhouette Score	0.441
Varianza explicada PCA (2D)	76.0%

Perfiles de negocio descubiertos:

Cluster	Perfil	Productos	Estabilidad	Marca propia	Categorías dominantes
0	Estables de Precio Medio	1,467	99.8%	58.6%	Mixto
1	Marcas Nacionales en Promoción	120	Baja (rango alto)	10.8%	Cuidado facial, bodega
2	Básicos Estables de Marca Blanca	2,942	99.8%	77.4%	Cuidado personal, limpieza
3	Frescos de Alta Volatilidad	30	41.8%	0.0%	Fruta/verdura, pescado
4	Dinámicos de Variabilidad Moderada	490	96.2%	26.7%	Mixto



Explicación: Visualización bidimensional del clustering K-Means proyectado mediante PCA (Componentes Principales 1 y 2). Las marcas 'X' representan los centroides de cada cluster en el espacio de menor dimensión, delimitando la separación espacial del catálogo.

Proyección PCA bidimensional (78.8% de varianza explicada) mostrando los 5 perfiles. El Cluster 3 (Frescos de Alta Volatilidad, 30 productos) se separa radicalmente del resto, mientras que los Clusters 0 y 2 forman una masa compacta de productos estables.

Rotación del catálogo

Evento	Cantidad
Productos nuevos detectados	615
Productos descatalogados confirmados	717

Shrinkflation

Métrica	Valor
Casos únicos confirmados	20
Variación media precio absoluto	-0.10%
Variación media precio por medida	+12.39%
Severidad media	12.49

API Reference

Método + Ruta	Descripción	Parámetros	Campos clave respuesta
GET /api/categorias	Lista categorías y subcategorías	—	categorias: {cat: [subcat, ...]}
GET /api/productos	Productos filtrados del catálogo	categoria , subcategoria , marca , q , limit	total , productos: [{referencia, titulo, precio_actual, ...}]
GET /api/cestas	Perfiles de cesta predefinidos	—	perfiles: {familiar: {nombre, descripcion, n_productos, productos}}
POST /api/ipc	Calcula IPC ponderado de una cesta	Body: {productos: [{referencia, cantidad_mensual}]} O {perfil: "familiar"}	ipc_actual , variacion_total , fechas , ipc_cesta , por_producto
POST /api/ipc/prediccion	Predicción de coste futuro de la cesta	Body: igual que /ipc . Query: horizonte (días, max 90)	coste_actual , coste_predicho , variacion_esperada , desglose
POST /api/recomendaciones	Alternativas más baratas (NLP)	Body: igual que /ipc	ahorro_total_mensual , n_con_ahorro , recomendaciones
GET /api/equivalencias	Equivalencias NLP marca propia ↔ comercial	subcategoria , marca , min_similitud , limit	total , equivalencias: [{titulo_mp, titulo_com, similitud, diferencia_por_medida_pct}]
GET /api/anomalias/hoy	Productos anómalos en la última fecha	metodo : zscore , if , ae , todos	fecha , resumen: {zscore: N, ...} , anomalias
GET /api/catalogo/eventos	Productos nuevos y descatalogados	tipo : nuevo , descatalogado , todos . limit	nuevos: [{referencia, titulo, primera_fecha}] , descatalogados

Método + Ruta	Descripción	Parámetros	Campos clave respuesta
GET /api/shrinkflation	Alertas de shrinkflation	categoria , min_severidad , limit	total , alertas: [{referencia, titulo, severidad, reduccion_pct}]
GET /api/producto/<referencia>	Detalle completo de un producto	referencia (int, en URL)	producto , historial , anomalias , equivalencias , shrinkflation
GET /health	Healthcheck del sistema	—	status , productos , fecha_actual , predicciones_lstm

Ejemplo de petición

```
# Calcular IPC del perfil familiar
curl -X POST http://localhost:5000/api/ipc \
  -H "Content-Type: application/json" \
  -d '{"perfil": "familiar"}'

# Buscar productos por texto
curl "http://localhost:5000/api/productos?q=leche&marca=propia&limit=10"

# Predicción de coste a 30 días
curl -X POST http://localhost:5000/api/ipc/prediccion?horizonte=30 \
  -H "Content-Type: application/json" \
  -d '{"perfil": "estudiante"}'
```

Docker

El fichero `docker-compose.yml` orquesta cuatro servicios:

Servicio	Imagen	Puerto	Función
es01	elasticsearch:7.17.29	9200	Motor de búsqueda e indexación. Single-node con healthcheck.
kibana	kibana:7.17.29	5601	Exploración visual de datos y dashboards. Depende de ES.
api	Build desde <code>Dockerfile.api</code>	5000	API Flask. Monta <code>data/</code> y <code>models/</code> como volúmenes para persistencia sin rebuild.
frontend	Build desde <code>Dockerfile.frontend</code>	80	Frontend Vue servido con Nginx.

¿Por qué multi-stage build para el frontend? El `Dockerfile.frontend` utiliza dos etapas: una primera con `node:20-alpine` para compilar la aplicación Vue con Vite (`pnpm build`), y una segunda con `nginx:alpine` que copia solo los estáticos generados (`dist/`). Esto reduce el tamaño de la imagen final de ~1 GB (con `node_modules`) a ~30 MB (solo HTML/CSS/JS estáticos + Nginx), mejorando drásticamente los tiempos de despliegue y el consumo de recursos.



Notebooks

Notebook	Descripción	Entorno
<code>anomalias_autoencoder_colab.ipynb</code>	Entrenamiento del Autoencoder LSTM con GPU T4 (642K secuencias normales)	Google Colab
<code>lstm_clasificador_colab.ipynb</code>	Entrenamiento del LSTM clasificador de cambios de precio (658K secuencias)	Google Colab
<code>clustering_kmeans.ipynb</code>	Segmentación K-Means del catálogo en 5 perfiles de negocio	Local
<code>verificacion_parquet.ipynb</code>	Verificación de integridad del parquet maestro consolidado	Local

Metodología SCRUM

Sprint	Objetivo	Entregables clave
Sprint 1	Infraestructura de datos	Scraper GitHub Actions, pipeline ETL (<code>consolidar_historico.py</code> , <code>ingesta_incremental.py</code>), Parquet particionado, indexación en Elasticsearch
Sprint 2	Detección de anomalías	Z-Score Rolling, Isolation Forest, Autoencoder LSTM (Colab), indexación de anomalías en ES, dashboards Kibana
Sprint 3	Predicción y NLP	LSTM Clasificador (Colab), pipeline NLP sentence-transformers, equivalencias marca propia ↔ comercial, API Flask, IPC personalizado
Sprint 4	Predicción avanzada y frontend	XGBoost Regresor con integración LSTM, análisis SHAP, walk-forward validation, frontend Vue 3 completo
Sprint Plus 1	Análisis complementarios	Detector de shrinkflation, detector de rotación del catálogo (altas/bajas), indexación en ES
Sprint Plus 2	Pulido y despliegue	Docker Compose con 4 servicios, multi-stage build, dashboards Kibana exportados, documentación de análisis completa

Limitaciones conocidas

NLP y equivalencias semánticas

- **Equivalencia semántica ≠ equivalencia funcional.** El matching por texto no distingue productos que comparten vocabulario pero tienen funciones distintas (lavavajillas de uso diario vs limpiamáquinas).
- **Sesgo hacia la marca comercial disponible.** En subcategorías con pocos productos comerciales, todos los matches se concentran en 1-2 productos no representativos. Se mitiga con `MIN_COMERCIALES ≥ 3`.
- **Sensibilidad al denominador.** El cálculo porcentual amplifica diferencias cuando el precio/medida de marca propia es muy bajo. Se mitiga con la mediana como estadístico principal.

Clustering K-Means

- **Sesgo por geometría esférica.** K-Means asume clusters esféricos (distancias euclidianas), lo que puede forzar divisiones artificiales en distribuciones alargadas.
- **Dependencia de la ventana temporal.** Las variables de volatilidad dependen de la ventana de 154 días analizada; en periodos macroeconómicos estables, algunos clusters podrían fusionarse.

XGBoost

- **Persistencia imbatible globalmente.** En series con estructura de escalón, la predicción trivial (`precio_mañana = precio_hoy`) es casi óptima ($R^2 = 0.9984$). XGBoost solo aporta valor marginal en el 4.1% de casos con cambio real.
- **Sesgo hacia productos baratos.** El 95% del catálogo tiene precios <30€; el modelo comete errores severos en productos caros (400-500€).
- **Degradación temporal.** El modelo entrenado con datos estables (noviembre-marzo) se degrada significativamente cuando los patrones de precio cambian (abril-mayo).

Shrinkflation

- **"Falso amigo" del producto fresco.** En productos vendidos por pieza (pescadería, carnicería), el peso varía naturalmente según el lote, pudiendo generar alertas no vinculadas a una estrategia comercial deliberada.
- **100% de alertas en producto fresco.** La reduflación detectada se concentra en fruta/verdura (85%) y marisco/pescado (15%), todas marcas comerciales. No se detectan casos en productos industriales envasados, lo cual puede deberse a la ventana temporal analizada.

Detector de catálogo

- **Falsa baja por cambio de referencia.** Si un fabricante cambia el ID numérico del producto manteniendo título y formato, el sistema interpreta una baja y un alta simultáneas, sobreestimando la rotación.
- **Dependencia del lineal local.** El scraper opera con un código postal concreto; las bajas pueden reflejar cambios de stock regionales, no decisiones a nivel nacional.

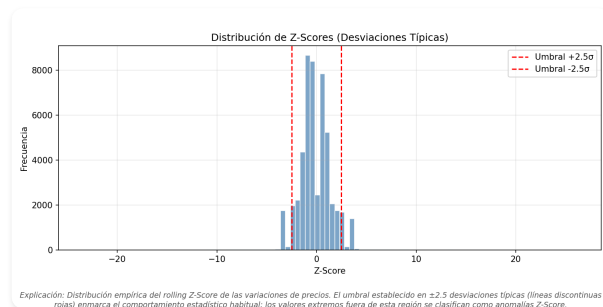


Galería de visualizaciones

Todas las visualizaciones se generan automáticamente durante el entrenamiento y evaluación de los modelos. Los archivos fuente se encuentran en `docs/img/`.

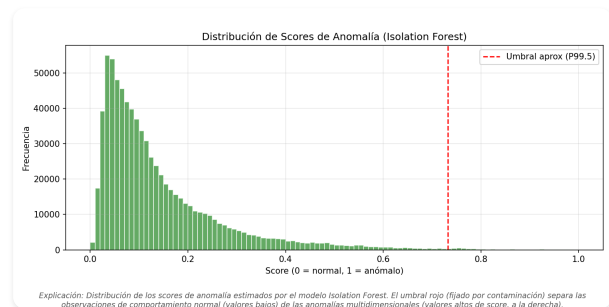
Detección de anomalías

Z-Score — Distribución de desviaciones típicas



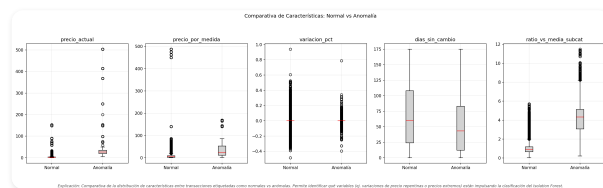
Las líneas rojas en $\pm 2.5\sigma$ delimitan el umbral: todo valor fuera de esta región es una anomalía temporal.

Isolation Forest — Distribución de scores



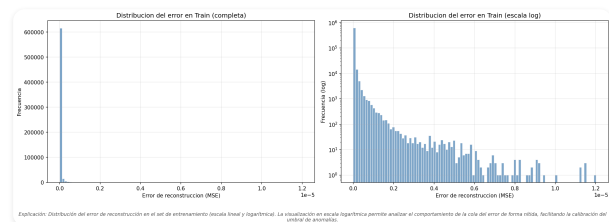
El umbral (línea roja, P99.5) separa las observaciones normales (scores bajos) de las anomalías multidimensionales (scores altos).

Isolation Forest — Comparativa de features: Normal vs Anomalía



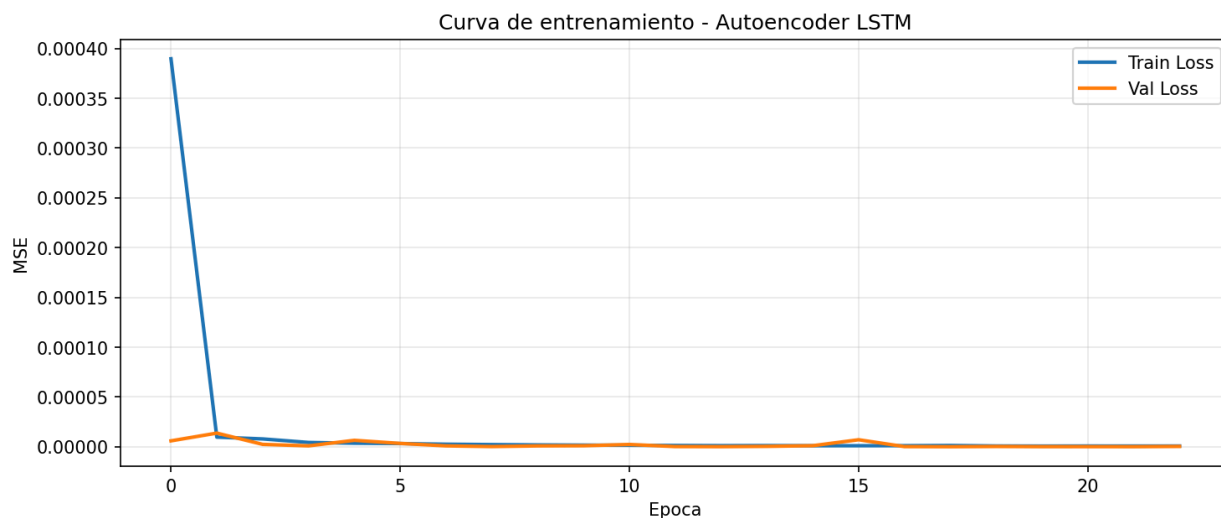
Las anomalías IF presentan un `ratio_vs_media_subcat` extremo ($2x-11x$), confirmando que son productos estructuralmente premium en subcategorías baratas.

Autoencoder LSTM — Distribución del error de reconstrucción



La distribución bimodal (escala log, derecha) confirma que el Autoencoder separa eficazmente secuencias estables (error ≈ 0) de secuencias con cambios de precio (cola derecha).

Autoencoder LSTM — Curvas de entrenamiento (Google Colab)



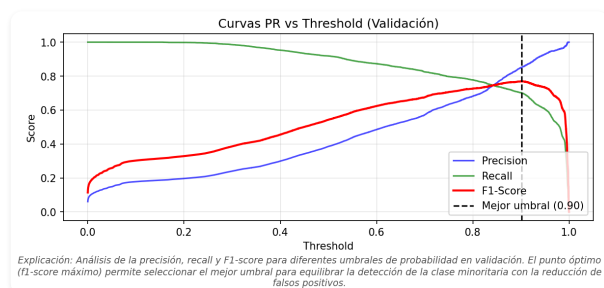
Explicación: Curvas de pérdida de entrenamiento y validación (MSE) en Google Colab. La convergencia y estabilización de ambas curvas a lo largo de las épocas valida el correcto aprendizaje del autoencoder sobre secuencias temporales.

Convergencia rápida del MSE en entrenamiento y validación (GPU T4 en Colab). La estabilización tras la época 5 confirma que el modelo no sobreajusta a los patrones normales.

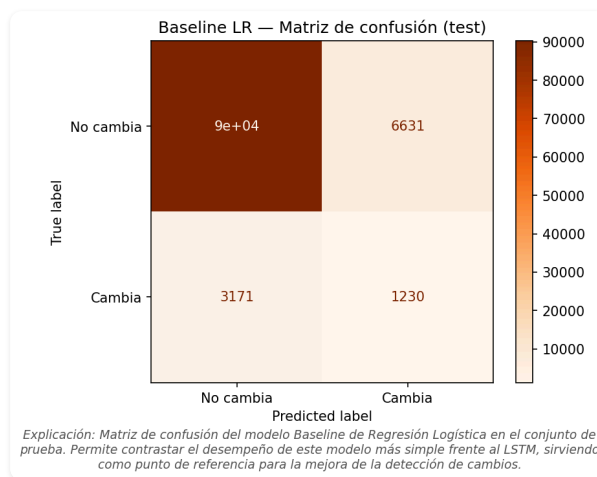
LSTM Clasificador — Optimización del umbral

Baseline (Regresión Logística) — Matriz de confusión

Curvas Precision-Recall vs Threshold

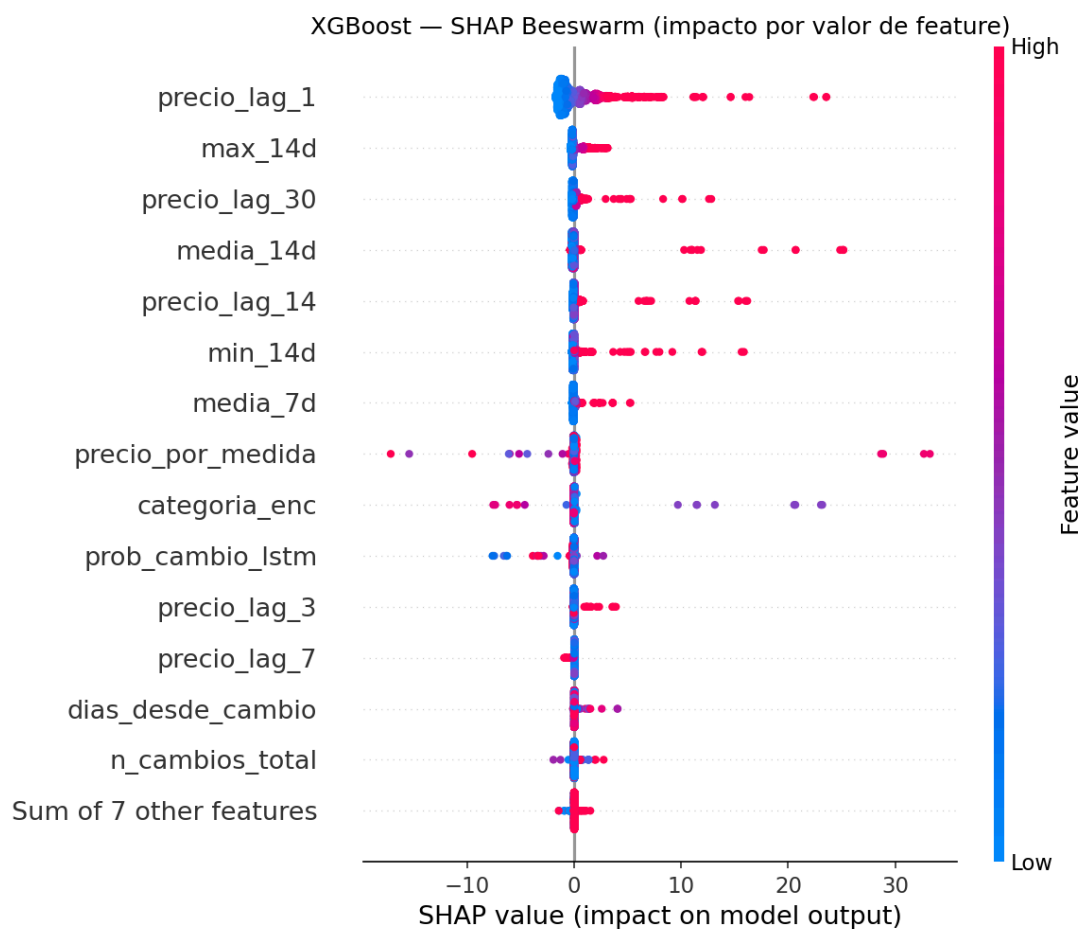


El umbral óptimo (0.90, línea negra) maximiza el F1-Score en validación, equilibrando la detección de cambios reales con la minimización de falsas alarmas.



La Regresión Logística detecta solo 1,230 de 4,401 cambios (Recall 27.9%), confirmando que el aplanamiento de los datos destruye las interacciones temporales captadas por el LSTM.

XGBoost — Explicabilidad SHAP



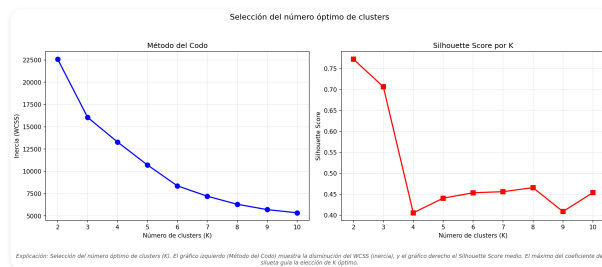
Explicación: Gráfico Beeswarm de valores SHAP. Cada punto representa un producto en el conjunto de test. La posición en el eje X indica si la característica aumenta (derecha) o disminuye (izquierda) la predicción del precio futuro. El color rojo indica valor alto y el azul valor bajo.

Beeswarm Plot SHAP: cada punto es una predicción del conjunto de test. El color indica el valor de la feature (rojo = alto, azul = bajo). Se observa que valores altos de `precio_lag_1` empujan la predicción al alza (puntos rojos a la derecha), mientras que `prob_cambio_lstm` actúa como modulador de incertidumbre.

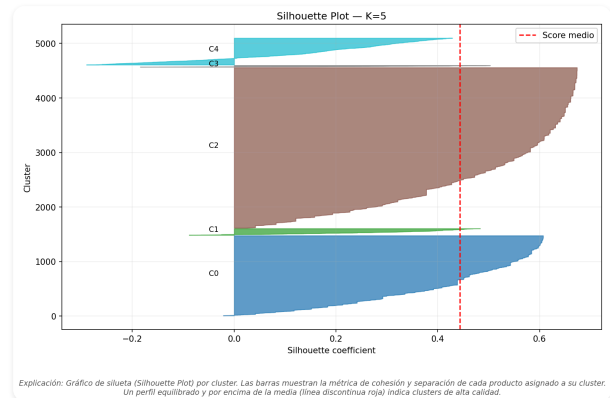
Clustering — Selección de K y validación

Silhouette Plot — Análisis de cohesión por cluster

Método del Codo + Silhouette Score por K

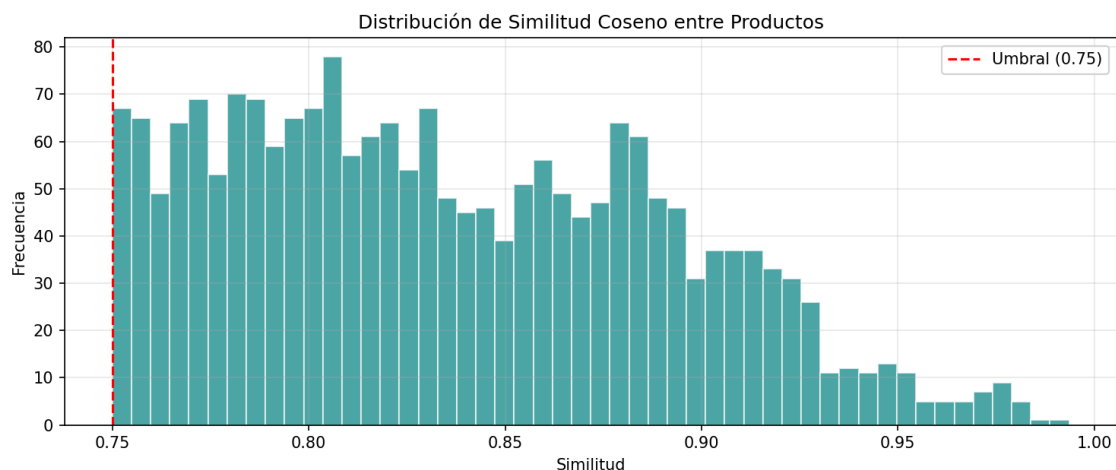


K=5 se selecciona en el punto de inflexión de inercia (izq.) y se valida con un Silhouette Score competitivo de 0.441 (dcha.).



Los Clusters 0 y 2 muestran perfiles gruesos y simétricos (buena cohesión). El Cluster 1 (Marcas Nacionales en Promoción) tiene un perfil más delgado y valores negativos, reflejando la heterogeneidad inherente a los productos de marca comercial.

NLP — Distribución de similitud



Distribución del score de similitud coseno entre los 924 pares equivalentes. La densidad principal entre 0.78-0.90 indica matches de alta calidad, con una cola hasta 1.0 correspondiente a equivalencias casi exactas (ej. tomate frito Hacendado ↔ Hida, sim=0.96).



Autor

Daniel Porras Morales

Trabajo Fin de Estudios — Curso de Especialización en Inteligencia Artificial y Big Data
2025-2026



Licencia

MIT License — Daniel Porras Morales — 2026